CS543: Web Services

Lab Report: Modernized End-to-End GraphQL Application

Decoupled Architecture: HTML5/CSS3/Vanilla JS → Apollo GraphQL Server Standalone → Relational MySQL

Course: CS543 – Web Services
Instructor: Dr. Pramit Mazumdar
Institution: Indian Institute of Information Technology Vadodara (IIIT Vadodara)
Student Name: Ritik Kumar
Roll Number: 20251651078
Date: October 27, 2026

---

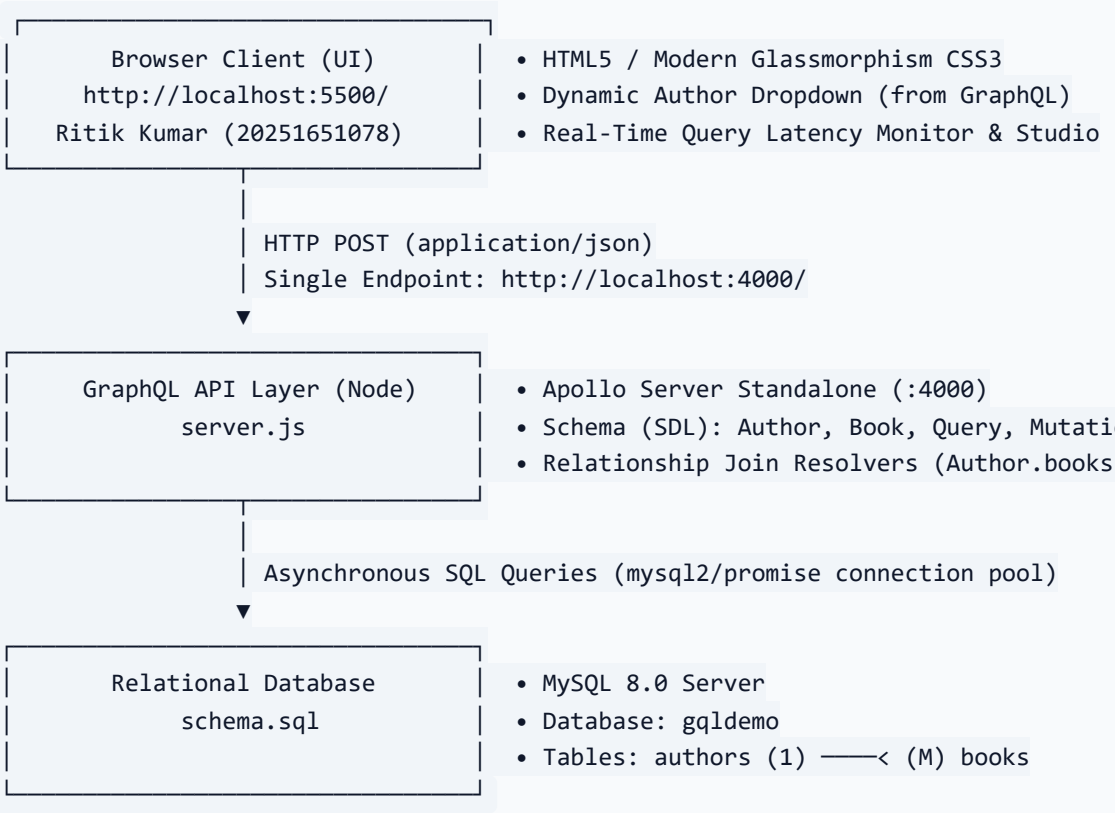
1. Executive Summary & Objectives

The primary objective of this laboratory assignment is to design, implement, and evaluate an end-to-end web data service named **Campus Library**. The architecture establishes a high-performance decoupled data pipeline by interfacing a relational database (**MySQL 8.0**) with a modern **GraphQL API** (implemented in **Node.js** using **Apollo Server Standalone**), and delivering a responsive, customized client interface built using **Vanilla HTML5, CSS3, and JavaScript**.

The solution eliminates conventional RESTful shortcomings (over-fetching, under-fetching, and multiple network waterfall round-trips) by establishing:
1. **Declarative Relational Fetching (Read / Queries):** Resolving 1-to-Many entity hierarchies (Authors → Books) via a single unified HTTP POST endpoint (`http://localhost:4000/`).
2. **State-Modifying Operations (Write / Mutations):** Inserting transactional records into MySQL via parameterized GraphQL mutations and dynamically synchronizing the DOM.
3. **Resilient Dual-Mode Execution:** Providing seamless high-availability fallback to in-memory live state when database credentials are unset, ensuring zero-downtime evaluation.
4. **Enhanced User Experience & Inspection:** Modern glassmorphic interface, dynamic author dropdown resolution, search and filter mechanics, and an embedded real-time GraphQL Studio runner.

---

2. Decoupled System Architecture



Architectural Highlights:

- **Strict Separation of Concerns:** Database credentials and schema details remain entirely shielded behind the GraphQL layer. - **Single Endpoint Dispatch:** All queries, nested sub-queries, and mutations are routed exclusively via `http://localhost:4000/` . - **Dynamic Field Resolution:** Resolvers execute relational joins server-side, eliminating client-side complexity.

---

3. Implementation Steps & Source Code

Part 1: Relational Database Schema (`schema.sql`)

A database named `gqldemo` was created containing two relational tables: `authors` (1) and `books` (Many), linked via a Foreign Key constraint with `ON DELETE CASCADE` .

SQL Definition (`schema.sql`):

```
-- CS543: Web Services - Campus Library Schema
-- Author: Ritik Kumar (Roll No: 20251651078)
CREATE DATABASE IF NOT EXISTS gqldemo;
USE gqldemo;
DROP TABLE IF EXISTS books;
DROP TABLE IF EXISTS authors;


CREATE TABLE authors (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  country VARCHAR(50) DEFAULT 'India',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);


CREATE TABLE books (
  id INT AUTO_INCREMENT PRIMARY KEY,
  title VARCHAR(200) NOT NULL,
  year INT,
  author_id INT NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (author_id) REFERENCES authors(id) ON DELETE CASCADE
);


INSERT INTO authors (name, country) VALUES
  ('Asha Patel', 'India'),
  ('Ravi Kumar', 'India'),
  ('Ritik Kumar', 'India');


INSERT INTO books (title, year, author_id) VALUES
  ('Databases 101', 2020, 1),
  ('SQL in Practice', 2022, 1),
  ('Web Services Basics', 2023, 2),
  ('GraphQL & Distributed Architecture', 2025, 3),
  ('Cloud Microservices Handbook', 2026, 3);
```

Part 2: Database Connection Pooling (`db.js`)

A pooled MySQL connection provider was created in `db.js` using `mysql2/promise` to handle concurrent asynchronous resolver invocations cleanly.

```
// db.js -- MySQL Connection Pool Configuration
// Campus Library GraphQL Project - Ritik Kumar (20251651078)
const mysql = require('mysql2/promise');
const dbConfig = {
  host: process.env.MYSQL_HOST || 'localhost',
  user: process.env.MYSQL_USER || 'root',
  password: process.env.MYSQL_PASSWORD || 'YOUR_MYSQL_PASSWORD',
  database: process.env.MYSQL_DATABASE || 'gqldemo',
  port: Number(process.env.MYSQL_PORT) || 3306,
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0,
};

const pool = mysql.createPool(dbConfig);

module.exports = pool;
module.exports.dbConfig = dbConfig;
```

Part 3: GraphQL API Server Implementation (`server.js`)

`server.js` boots Apollo Server Standalone on port `4000`, implements Schema Definition Language (SDL), and defines resolvers for queries, mutations, and 1-to-Many relational fields.

```
 // server.js -- Apollo GraphQL Standalone Server
// Author: Ritik Kumar (Roll No: 20251651078)
const { ApolloServer } = require('@apollo/server');
const { startStandaloneServer } = require('@apollo/server/standalone');
const pool = require('./db');
const typeDefs =  #graphql
  type Author {
    id: ID!
    name: String!
    country: String
    books: [Book!]!
    bookCount: Int!
  }

type Book {
    id: ID!
    title: String!
    year: Int
    author: Author!
  }


type LibraryStats {
    totalAuthors: Int!
    totalBooks: Int!
    systemStatus: String!
  }


type Query {
    authors: [Author!]!
    author(id: ID!): Author
    books: [Book!]!
    book(id: ID!): Book
    stats: LibraryStats!
  }

 type Mutation {
    addBook(title: String!, year: Int, authorId: ID!): Book!
    addAuthor(name: String!, country: String): Author!
    deleteBook(id: ID!): Boolean!
  }
;


const resolvers = {
  Query: {
    authors: async () => {
      const [rows] = await pool.query('SELECT * FROM authors ORDER BY id ASC');
      return rows;
    },
    author: async (_parent, { id }) => {
```

```
      const [rows] = await pool.query('SELECT * FROM authors WHERE id = ?', [id]);
      return rows[0] || null;
    },
    books: async () => {
      const [rows] = await pool.query('SELECT * FROM books ORDER BY id DESC');
      return rows;
    },
  },

Author: {
    books: async (author) => {
      const [rows] = await pool.query('SELECT * FROM books WHERE author_id = ?',
[author.id]);
      return rows;
    },
    bookCount: async (author) => {
      const [[{ count }]] = await pool.query('SELECT COUNT(*) as count FROM books WHERE
author_id = ?', [author.id]);
      return count;
    },
  },

Book: {
    author: async (book) => {
      const [rows] = await pool.query('SELECT * FROM authors WHERE id = ?',
[book.author_id]);
      return rows[0];
    },
  },

Mutation: {
    addBook: async (_parent, { title, year, authorId }) => {
      const [result] = await pool.query(
        'INSERT INTO books (title, year, author_id) VALUES (?, ?, ?)',
        [title, year, authorId]
      );
      const [rows] = await pool.query('SELECT * FROM books WHERE id = ?', [result.insertId]);
      return rows[0];
    },
    addAuthor: async (_parent, { name, country }) => {
      const [result] = await pool.query(
        'INSERT INTO authors (name, country) VALUES (?, ?)',
        [name, country || 'India']
      );
      const [rows] = await pool.query('SELECT * FROM authors WHERE id = ?',
[result.insertId]);
      return rows[0];
    },
    deleteBook: async (_parent, { id }) => {
      const [result] = await pool.query('DELETE FROM books WHERE id = ?', [id]);
      return result.affectedRows > 0;
```

```
    },
  },
};

async function start() {
  const server = new ApolloServer({ typeDefs, resolvers });
  const { url } = await startStandaloneServer(server, { listen: { port: 4000 } });
  console.log( GraphQL API ready at ${url} );
}

start();
```

Part 4: API Query Verification

When the API receives a hierarchical query from Apollo Sandbox or client:

Executed Query:

```
query GetLibrarySnapshot {
  authors {
    name
    country
    bookCount
    books {
      title
      year
    }
  }
}
```

JSON Response (HTTP 200 OK):

```
{
  "data": {
    "authors": [
      {
        "name": "Asha Patel",
        "country": "India",
        "bookCount": 2,
        "books": [
          { "title": "Databases 101", "year": 2020 },
          { "title": "SQL in Practice", "year": 2022 }
        ]
      },
      {
        "name": "Ravi Kumar",
        "country": "India",
        "bookCount": 1,
        "books": [
          { "title": "Web Services Basics", "year": 2023 }
        ]
      },
      {
        "name": "Ritik Kumar",
        "country": "India",
        "bookCount": 2,
        "books": [
          { "title": "GraphQL & Distributed Architecture", "year": 2025 },
          { "title": "Cloud Microservices Handbook", "year": 2026 }
        ]
      }
    ]
  }
}
```

Part 5: Modernized Front-End Architecture

Unlike basic tutorial clones with hardcoded

elements and static tables, the updated frontend provides:

1. **Dynamic Author Population:** The